

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I _Bharath Sankar (192425046)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: bharath s

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Answer:

1. Theoretical Concepts

Real-world images acquired through cameras or scanners are corrupted by noise arising from sensor limitations, low illumination and transmission artefacts (commonly modelled as Gaussian noise or salt-and-pepper noise). Feature detectors such as Harris corners, SIFT and ORB rely on local intensity gradients, so noise creates spurious gradients that are wrongly detected as corners or edges. Preprocessing therefore has two complementary goals:

- Noise removal - smoothing filters (Gaussian blur, median filter, bilateral filter) suppress high-frequency noise while trying to preserve genuine edges.
- Normalization - techniques such as min-max scaling and histogram equalization redistribute pixel intensities across the full dynamic range (0-255), improving contrast so that weak but genuine edges/corners become detectable.

The median filter was selected over the mean filter for impulse (salt-and-pepper) noise because it replaces a pixel with the median of its neighbourhood instead of the average, which removes outlier pixels without blurring edges as much. Histogram equalization was chosen for normalization because it is adaptive to the image's own intensity distribution, unlike a fixed linear stretch.

2. Implementation (Python 3 + OpenCV + NumPy + Matplotlib)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image and convert to grayscale
img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 2. Add synthetic Gaussian noise to simulate a noisy capture
noise = np.random.normal(0, 25, gray.shape).astype(np.float32)
noisy = np.clip(gray.astype(np.float32) + noise, 0, 255).astype(np.uint8)

# 3. Noise removal
gaussian_denoised = cv2.GaussianBlur(noisy, (5, 5), 0)
median_denoised = cv2.medianBlur(noisy, 5)

# 4. Normalization
minmax_norm = cv2.normalize(median_denoised, None, 0, 255, cv2.NORM_MINMAX)
hist_eq = cv2.equalizeHist(median_denoised)
```

```
# 5. Feature detection before vs after preprocessing (Harris corners)
def harris_keypoints(im):
    im32 = np.float32(im)
    dst = cv2.cornerHarris(im32, 2, 3, 0.04)
    return np.sum(dst > 0.01 * dst.max())

print('Keypoints - noisy image      :', harris_keypoints(noisy))
print('Keypoints - after median filter:', harris_keypoints(median_denoised))
print('Keypoints - after hist. eq.   :', harris_keypoints(hist_eq))

cv2.imwrite('output_noisy.png', noisy)
cv2.imwrite('output_median.png', median_denoised)
cv2.imwrite('output_histeq.png', hist_eq)
```

3. Workflow and Outputs

- Step 1: Load the reference image and convert to grayscale.
- Step 2: Inject synthetic Gaussian noise to emulate a noisy sensor acquisition.
- Step 3: Apply Gaussian blur and median filtering separately and save the denoised outputs.
- Step 4: Apply min-max normalization and histogram equalization on the denoised image.
- Step 5: Run Harris corner detection on the noisy image and on each preprocessed variant and record the keypoint counts.
- Output artefacts: noisy image, Gaussian-denoised image, median-denoised image, min-max normalized image, histogram-equalized image, and a console/table log of keypoint counts for each stage.

4. Analysis

The noisy image produced a very large number of Harris keypoints, most of which were false positives caused by random intensity spikes rather than genuine corners. After median filtering the keypoint count dropped sharply and the surviving keypoints corresponded closely to real object corners, confirming that median filtering is effective against impulse noise while preserving edge sharpness. Gaussian blurring also reduced false keypoints but, with a larger kernel, blurred and eliminated some genuine fine-detail corners, illustrating the classic smoothing-vs-detail trade-off. Histogram equalization on the denoised image increased local contrast in dark/low-contrast regions and revealed additional true corners that were invisible before normalization, but it also slightly amplified residual noise in flat regions. Overall, the combination of median filtering (for noise removal) followed by histogram equalization (for normalization) gave the most reliable feature-detection accuracy - fewer false positives than the raw noisy image and more true positives than denoising alone, showing that preprocessing choice and parameter tuning directly influence downstream feature-detection reliability.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Answer:

1. Theoretical Concepts

A colour image stores three channels (Red, Green, Blue), each an 8-bit intensity plane. Grayscale conversion collapses these three channels into a single luminance channel using a weighted sum that reflects human perceptual sensitivity to each colour:

$$\text{Gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

A binary image goes one step further: every grayscale pixel is compared with a threshold T and mapped to either 0 (black) or 255 (white), producing a two-level image that isolates foreground shapes from the background. The threshold can be a fixed global value, an automatically computed value (Otsu's method, which minimises intra-class intensity variance), or a locally adaptive value that handles uneven illumination.

Grayscale and binary representations are significant in computer vision because most classical detectors (Sobel, Canny, Harris, SIFT, ORB) are defined mathematically on a single intensity channel and operate on intensity gradients; working on a single channel also reduces memory and computation by a factor of three compared to colour. Binary images are further used for shape analysis, contour extraction, morphological operations (erosion/dilation) and object counting, where only the silhouette of an object matters.

2. Implementation (Python 3 + OpenCV)

```
import cv2

img = cv2.imread('sample.jpg')

# Grayscale conversion
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Binary conversion using Otsu's automatic thresholding
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Adaptive thresholding for uneven illumination
adaptive = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY, 11, 2)

# Edge detection and keypoint extraction on each representation
edges_gray = cv2.Canny(gray, 100, 200)
edges_binary = cv2.Canny(binary, 100, 200)

orb = cv2.ORB_create()
kp_gray, _ = orb.detectAndCompute(gray, None)
kp_binary, _ = orb.detectAndCompute(binary, None)
print('ORB keypoints on grayscale:', len(kp_gray))
print('ORB keypoints on binary :', len(kp_binary))

cv2.imwrite('gray.png', gray)
cv2.imwrite('binary.png', binary)
cv2.imwrite('adaptive.png', adaptive)
```

3. Workflow and Outputs

- Step 1: Load the source RGB image.
- Step 2: Convert to grayscale using the luminosity formula and save the output.
- Step 3: Convert the grayscale image to a binary image using Otsu's global threshold and, separately, an adaptive threshold; save both outputs.
- Step 4: Run Canny edge detection and ORB keypoint extraction on the grayscale and binary images.
- Output artefacts: grayscale image, Otsu binary image, adaptive-threshold binary image, edge maps for both, and keypoint counts.

4. Analysis

Edge detection on the grayscale image produced continuous, gradient-based edges that captured both strong object boundaries and subtle texture variations, because Canny operates on the full range of intensity gradients. Edge detection performed directly on the binary image produced edges only at the single black/white boundary created by the threshold; weak or low-contrast transitions that fall on the same side of the threshold were completely lost, and thin or narrow structures were sometimes merged or removed. Similarly, ORB feature extraction returned substantially fewer keypoints on the binary image than on the grayscale image, since ORB relies on fine-grained local gradient/intensity patterns that thresholding destroys. This confirms that grayscale representation preserves the gradient information required for accurate feature detection and matching, while binary representation is best suited to shape/contour-based tasks rather than gradient-based feature extraction, and premature binarization can therefore degrade downstream computer-vision performance.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Answer:

1. Theoretical Concepts

The Sobel operator estimates the first-order intensity gradient in the horizontal and vertical directions using two 3x3 convolution kernels, G_x and G_y . The gradient magnitude and direction are computed as:

$$G = \sqrt{G_x^2 + G_y^2}$$
$$\theta = \text{atan2}(G_y, G_x)$$

Sobel is simple and fast but is sensitive to noise and produces relatively thick, sometimes discontinuous edges because it has no built-in thresholding or thinning stage.

The Canny algorithm is a multi-stage procedure designed to overcome these limitations: (1) Gaussian smoothing to reduce noise, (2) gradient computation (internally similar to Sobel), (3) non-maximum suppression to thin the edges to single-pixel width along the direction of the gradient, and (4) double thresholding with hysteresis, which links weak edge pixels to strong edge pixels only if they are connected,

discarding isolated weak responses. This makes Canny more robust to noise and produces thin, well-localised, well-connected edges, at the cost of higher computational complexity and dependence on the choice of the two threshold values.

2. Implementation (Python 3 + OpenCV)

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel edge detection (combine horizontal and vertical gradients)
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx, sobely)
sobel = cv2.convertScaleAbs(sobel)

# Canny edge detection
canny = cv2.Canny(gray, 100, 200)

cv2.imwrite('sobel_edges.png', sobel)
cv2.imwrite('canny_edges.png', canny)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(gray, cmap='gray'); ax[0].set_title('Original (Gray)')
ax[1].imshow(sobel, cmap='gray'); ax[1].set_title('Sobel')
ax[2].imshow(canny, cmap='gray'); ax[2].set_title('Canny')
for a in ax: a.axis('off')
plt.savefig('sobel_vs_canny.png', dpi=150)
```

3. Workflow and Outputs

- Step 1: Load the input image and convert to grayscale so both operators work on the same intensity data.
- Step 2: Apply the Sobel operator in the X and Y directions and combine them into a gradient-magnitude edge map.
- Step 3: Apply the Canny operator with the same source image using fixed low/high hysteresis thresholds.
- Step 4: Save both edge maps and display them side by side with the original for visual comparison.
- Output artefacts: Sobel edge map, Canny edge map, and a combined comparison figure.

4. Analysis

The Sobel output showed thicker, somewhat noisy edges with visible responses even in low-texture background regions, because it lacks a non-maximum-suppression or hysteresis stage to discard weak, spurious gradients. The Canny output produced noticeably thinner, more continuous and better-localised edges around actual object boundaries, with background noise largely suppressed due to the initial Gaussian smoothing and the double-threshold hysteresis linking. On images with uniform lighting and clear boundaries, both methods identified the same major contours, but Canny more reliably preserved thin/weak-but-genuine edges through hysteresis, while Sobel either missed them (if a global threshold

was applied afterward) or produced excessive false edges (if not thresholded). In terms of computational cost, Sobel is considerably cheaper and is preferable for real-time or resource-constrained applications where approximate edges suffice, whereas Canny is preferable whenever precise, clean, single-pixel-wide edge maps are required for subsequent tasks such as contour extraction or object recognition, despite its higher computational cost and its sensitivity to the chosen threshold values.